

Ingressa

Venda de ingressos que não vende o mesmo lugar duas vezes

Análise, decisões e evolução: Júnior → Pleno → Sênior

Como ler este documento

Este é um estudo de caso. Ele não descreve só o que o sistema faz: explica **por que** cada decisão foi tomada, o que foi descartado e o que cada versão ensinou. A história segue esta linha:

Eu tinha uma lista de ideias → escolhi uma com um problema real → construí a versão mais simples que funciona (Júnior) → provei onde ela quebra → reorganizei e corriji (Pleno) → preparei para pico de acesso, falhas e operação real (Sênior).

Parte	Conteúdo
1. Visão geral	Nome, problema, solução, público e objetivo
2. Análise da ideia	Por que esta ideia, o que havia de errado nela e o que mudou
3. Análise técnica	Stack, arquitetura, banco, API, autenticação, segurança, infraestrutura, testes
4. Versão Júnior	O que foi construído e onde ela quebra
5. Versão Pleno	Como cada limitação foi resolvida
6. Versão Sênior	Escala, resiliência, segurança e operação: o que foi construído e medido
7. Decisões arquiteturais	Alternativas, vantagens, desvantagens e trade-offs
8. Roadmap	Do zero à versão avançada
9. Aprendizados	Conhecimentos desenvolvidos em cada etapa
10. Valor para o portfólio	O que o projeto demonstra para um recrutador
Apêndice	Como rodar e como validar

1. Visão geral

Item	Descrição
Nome	Ingressa
Descrição	Plataforma de venda de ingressos para eventos: organizadores cadastram eventos e setores; clientes reservam, pagam e recebem ingressos com código único.
Problema	Vendas concorridas derrubam sites, vendem o mesmo lugar duas vezes, perdem pagamentos no meio do caminho e favorecem robôs.
Solução	Um sistema que trata concorrência, consistência e justiça como requisitos centrais — e que evolui em três versões para mostrar cada técnica sendo introduzida quando passa a ser necessária.
Público-alvo	Produtoras de eventos pequenas e médias (vendedores) e o público que compra ingressos.
Objetivo	Servir de peça principal do portfólio: mostrar fundamentos (Júnior), organização profissional (Pleno) e raciocínio de arquitetura (Sênior) sobre o mesmo domínio.

Por que este problema é bom para aprender

- Qualquer pessoa entende o problema em uma frase ("não vender o mesmo lugar duas vezes").
- Por trás da frase simples existem desafios clássicos de backend: condição de corrida, transações, consistência entre sistemas, filas e picos de acesso.
- É fácil demonstrar o problema e a solução com números — o que torna a evolução verificável, não só descrita.

2. Análise da ideia

Origem

A ideia veio do item 27 da lista "27 projetos para ir de jr a sênior em programação web": *"Sistemas fullstack com filas: sistemas de leilão, sistemas de venda de ingresso. Tente usar filas, microserviços etc."* A análise completa das 27 ideias está no documento 00 da pasta de documentação geral.

Por que esta ideia foi escolhida

Critério	Nota (1-5)	Justificativa
Dificuldade técnica	5	Concorrência, consistência, mensageria, escala
Valor para portfólio	5	Conversa diretamente com vagas de backend .NET
Potencial comercial	3	Mercado com concorrentes fortes (Sympla, Eventim), mas nicho de produtoras pequenas existe
Encaixe com a carreira	5	C#/.NET, SQL, cloud e segurança — as áreas prioritárias

Problemas encontrados na ideia original

Problema	Por que é um problema	Alteração feita
"Tente usar microserviços"	Começar por microserviços troca uma transação simples por uma saga distribuída. É o erro mais comum em projetos de estudo.	Monólito em camadas + Worker separado. Microserviços só na Sênior, e só se uma medição justificar.
Escolher entre leilão e ingressos	Leilão exige tempo real e regras de lance que desviam do foco.	Ingressos: regra simples de explicar, problema muito conhecido no Brasil.
Pagamento real	Exige contrato com gateway e cuidados de PCI DSS.	Gateway simulado com o mesmo contrato de um real (token, aprovação, recusa, estorno).

Problema	Por que é um problema	Alteração feita
Mapa de assentos	Muito trabalho de interface para pouco aprendizado de backend.	Setores com capacidade. Assento numerado fica como evolução.
"Com filas" desde o início	Filas não fazem sentido antes de existir algo assíncrono.	Júnior sem filas; filas entram na Pleno junto com o pagamento e a emissão.

Ideias combinadas

Outras ideias da lista viraram funcionalidades do Ingressa, em vez de projetos separados: **lista, busca, ordenação e paginação** (vitrine), **tela de login/cadastro** (autenticação), **formulários complexos** (cadastro de evento e setores) e **e-commerce** (reserva e pagamento).

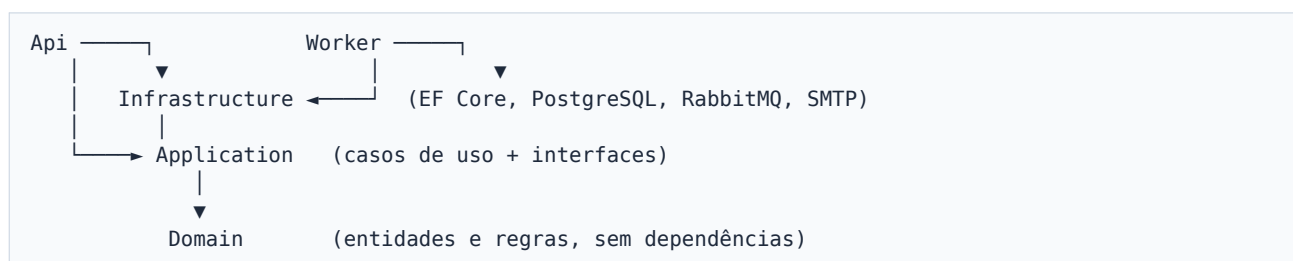
3. Análise técnica

Stack escolhida e justificativa

Tecnologia	Onde	Por quê
C# 14 / .NET 10	Todas as versões	Prioridade de carreira; versão com suporte de longo prazo (LTS); continuidade do projeto helpdesk.
ASP.NET Core (controllers)	API	Organização explícita para quem está começando; atributos de autorização por endpoint; OpenAPI nativo.
Entity Framework Core 10	Dados	ORM padrão do ecossistema; migrations; LINQ traduzido para SQL; ExecuteUpdate para operações atômicas.
SQLite → PostgreSQL 17	Júnior → Pleno	SQLite: zero instalação. PostgreSQL: concorrência real, xmin, índices parciais, SKIP LOCKED, jsonb.
JWT (HS256)	Autenticação	Padrão de mercado para APIs; na Pleno, combinado com refresh token em cookie.
RabbitMQ 4	Pleno e Sênior	Broker mais usado com .NET; filas quorum, dead letter, publisher confirms.
Redis 7 (+ scripts Lua)	Sênior	Fila virtual, limites distribuídos e cache: estruturas em memória com execução atômica entre réplicas.
OpenTelemetry	Sênior	Padrão aberto de traces e métricas; o destino (Grafana, X-Ray, CloudWatch) muda por configuração.
k6 e Playwright	Sênior	Carga com metas que reprovam a execução; fluxo de compra verificado no navegador, inclusive no celular.
Terraform (AWS)	Sênior	Ambiente de produção versionado e revisável; ECS Fargate, RDS, ElastiCache, Amazon MQ, WAF.
React 19 + TypeScript	Interface Pleno e Sênior	Tipagem dos contratos da API; ecossistema mais pedido em vagas full stack.
xUnit, Testcontainers, Vitest	Testes	Padrões do mercado; Testcontainers permite testar contra PostgreSQL real.
Docker Compose, GitHub Actions	Operação	Ambiente reproduzível com um comando; CI gratuito integrado ao GitHub.
Scalar	Documentação	Interface moderna sobre o OpenAPI gerado pelo próprio ASP.NET Core.

Arquitetura

Júnior: um projeto com pastas por responsabilidade (Controllers, Services, Models, Data, Auth, Erros). **Pleno:** camadas com dependências apontando para dentro — Domain não conhece ninguém; Application define interfaces; Infrastructure as implementa; Api e Worker montam tudo.



Banco de dados

Tabela	Papel	Detalhes importantes
Usuarios	Contas	E-mail único; perfil e status como texto; tentativas falhas e bloqueio
RefreshTokens	Sessões	Só o hash SHA-256; família para detectar reuso

Tabela	Papel	Detalhes importantes
Eventos / Setores	Catálogo e estoque	CHECK (Ocupados entre 0 e Capacidade); índice (Publicado, DataInicio)
Pedidos / ItensPedido	Reservas e compras	Preço congelado no item; xmin como versão; índice parcial das reservas pendentes
Ingressos	Entrada no evento	Código único de 64 bits aleatórios
MensagensOutbox	Mensageria confiável	jsonb; índice parcial das pendentes; tentativas e último erro

APIs, autenticação e autorização

- API REST com Problem Details (RFC 9457) para erros: 400 validação, 401 sessão, 403 perfil/dono, 404 inexistente ou alheio, 409 conflito de estado, 422 regra de negócio, 429 excesso de tentativas.
- **Autenticação:** Júnior com JWT de 60 min; Pleno com access token de 15 min em memória e refresh token rotativo em cookie HttpOnly/Secure/SameSite=Strict.
- **Autorização:** por perfil (Cliente, Organizador, Admin) e por propriedade (só o dono altera o evento; pedido de outra pessoa responde 404). Na Pleno, organizadores precisam de aprovação.

Segurança, infraestrutura e testes

Os detalhes estão nas seções de cada versão. Em resumo: hash PBKDF2 com 600 mil iterações, comparação em tempo constante, bloqueio de conta, limite por IP, escape de curingas no ILIKE, cabeçalhos de segurança, CSP, contêineres sem privilégios e nenhum segredo no repositório. A infraestrutura sobe com Docker Compose (PostgreSQL, RabbitMQ, Mailpit, API, Worker, nginx). Os testes vão de unidade (domínio puro) a integração com PostgreSQL real.

4. Versão Júnior — construir o simples, bem feito

Objetivo: provar fundamentos. Uma API correta, organizada, testada e documentada, que qualquer pessoa roda com um comando.

O que foi construído

Área	Entregue
Autenticação	Cadastro e login; PBKDF2 com salt; JWT com emissor, audiência e expiração; aplicação não inicia com chave fraca
Eventos	CRUD com rascunho e publicação; setores com preço e capacidade; só o dono altera; eventos com vendas não são excluídos
Vitrine	Busca, filtro por cidade, ordenação e paginação estável
Compra	Até 6 ingressos; validação de todos os itens antes de alterar; preço congelado; código aleatório; cancelamento até 24 h antes
Erros	Exceções de domínio convertidas em Problem Details em um único lugar
Interface	Vitrine em JavaScript puro com escape de HTML (proteção contra XSS)
Qualidade	54 testes (xUnit) com SQLite em memória real e relógio controlado; 11 commits no padrão Conventional Commits; README com diagramas

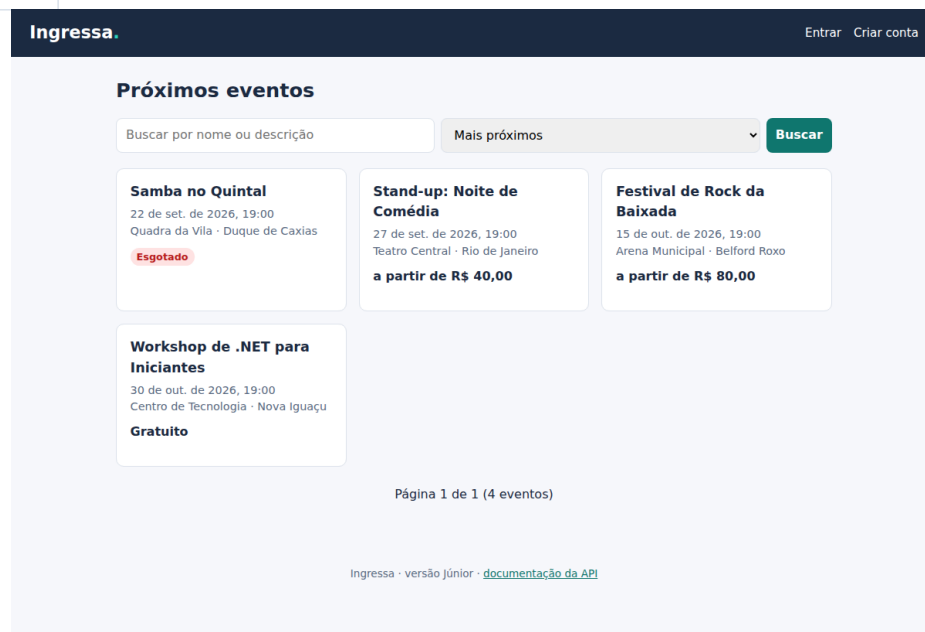


Figura 1 — Vitrine da versão Júnior.

Onde a versão Júnior quebra

A baixa de estoque segue o padrão "ler → calcular → gravar". Para demonstrar o impacto, o mesmo padrão foi executado diretamente no PostgreSQL com 200 compras simultâneas para 10 lugares:

Método	Compras aceitas	Lugares registrados	Resultado
Júnior: ler → calcular → gravar	200	9	190 ingressos vendidos sem lugar; contador errado
Pleno: UPDATE condicional	10	10	Correto

A limitação foi mantida de propósito e documentada no código e no README. Ela é o motivo da versão Pleno existir. Outras limitações: compra sem reserva nem pagamento, tudo em um

projeto, token sem renovação, qualquer pessoa se cadastrando como organizador, esquema criado sem migrations, e ausência de Docker e CI.

5. Versão Pleno — estruturar como um sistema profissional

Objetivo: resolver cada limitação da Júnior com a técnica adequada — e não adicionar nada sem motivo.

Limitação	Solução	Por que esta e não outra
Venda duplicada	UPDATE atômico condicional + CHECK no banco	Não segura travas; nenhuma tentativa válida precisa ser repetida (ADR 0005)
Pagar e expirar ao mesmo tempo	Concorrência otimista com xmin	Conflito raro: melhor detectar do que travar
Sem reserva/pagamento	Reserva de 10 min, gateway simulado, expiração em lote, estorno automático	Modelo usado pelas grandes plataformas
Mensagem perdida se o processo cair	Transactional Outbox + RabbitMQ	Elimina a escrita dupla (ADR 0006)
Tudo em um projeto	Camadas + Worker	Domínio testável sem banco; transação única preservada (ADR 0004)
Token exposto e irrevogável	Refresh token rotativo em cookie HttpOnly, com detecção de reuso	Recomendação do OAuth 2.0 Security BCP (ADR 0007)
Força bruta	Bloqueio de conta + limite por IP	Duas camadas: por conta e por origem
Organizadores falsos	Aprovação por administrador	Quem vende responde pelo evento
EnsureCreated / SQLite	PostgreSQL + migrations + checagem no CI	Esquema versionado e revisável
Sem operação	Docker Compose, CI, logs JSON correlacionados, health checks	Ambiente idêntico para qualquer pessoa

O fluxo de uma compra

```

Cliente —POST /pedidos→ API —BEGIN
    UPDATE Setores SET Ocupados = Ocupados + n
    WHERE Id = @s AND Capacidade - Ocupados >= n    (por setor)
    INSERT Pedido (AguardandoPagamento, expira em 10 min)
    COMMIT → 201
Cliente —POST /pagamento→ API → gateway (fora da transação)
    UPDATE Pedido ... WHERE xmin = @versao + INSERT outbox "pedido.pago"
"
Worker —SELECT outbox FOR UPDATE SKIP LOCKED→ publica no RabbitMQ (com confirmação)
Worker ← "pedido.pago" — emite ingressos (idempotente) + outbox "ingressos.emitidos"
Worker ← "ingressos.emitidos" — envia e-mail com os códigos
Worker (a cada 30 s) — reservas vencidas ► Expirado + devolve os lugares

```

Figura 2 — Pagamento recusado; a reserva continua valendo.

Setor	Código do ingresso	Valor
Pista	1A1908C3AE60000	R\$ 80,00
Pista	E25403AB23EB2000	R\$ 80,00
Pista Premium	874604FE16E71000	R\$ 150,00
Total: R\$ 310,00		

Figura 3 — Ingressos emitidos pelo Worker.

Casos difíceis tratados

- **Reserva expira enquanto o gateway processa:** o pagamento foi aprovado, mas o pedido já não pode ser pago. O valor é estornado automaticamente e o cliente recebe 422.
- **Um setor do pedido não tem lugar:** a transação inteira é desfeita, inclusive os setores que já tinham sido ocupados.
- **Mensagem entregue duas vezes:** a emissão verifica se já emitiu; a segunda entrega não faz nada.
- **Mensagem que sempre falha:** após 5 entregas vai para a fila de falhas, sem travar as demais.
- **Refresh token copiado por um atacante:** quando o token já usado reaparece, toda a família de tokens é revogada.
- **Usuário digita % na busca:** o curinga é escapado; a busca procura o caractere literal.

Estratégia de testes

Nível	Quantidade	Ferramenta	O que garante
Domínio	43	xUnit	Regras e máquina de estados, sem banco
Casos de uso	34	xUnit + adaptadores em memória	Orquestração, rollback, 50 reservas paralelas, estorno, rotação de sessão
Integração	15	WebApplicationFactory + Testcontainers	API real contra PostgreSQL real: 40 clientes x 5 lugares, xmin, cookies, 429
Interface	11	Vitest + Testing Library	Renovação automática e única da sessão, mensagens de erro, vitrine

O que foi deixado de fora de propósito

MediatR, AutoMapper, Redis, ASP.NET Core Identity e microserviços foram avaliados e não entraram **nesta versão**: com o tamanho do sistema naquele momento, cada um acrescentaria complexidade sem resolver um problema existente. A justificativa está no README da versão Pleno. O Redis entrou na Sênior quando surgiu o problema que ele resolve (fila virtual, limites entre réplicas e cache); os demais continuam fora.

6. Versão Sênior — escala, resiliência e operação

Pergunta desta versão: o sistema está correto, mas aguenta a abertura de vendas de um show disputado — e dá para operá-lo quando algo der errado às 2h da manhã?

Limitações da Pleno e o que foi feito

Limitação da Pleno	Solução na Sênior	Como foi verificado
Na abertura de vendas, todos disputam o banco ao mesmo tempo	Fila virtual por evento no Redis: sorted set com a ordem de chegada e passe de uso único; o Worker admite no máximo N compradores simultâneos	17 cenários dos scripts Lua contra um Redis real; teste de integração da fila ponta a ponta; k6 com 300 compradores
Clique duplo ou retentativa criava uma segunda reserva	Cabeçalho Idempotency-Key com tabela e índice único (usuário, chave); a repetição recebe a resposta original	Testes de integração: cinco cliques simultâneos criam um pedido; mesma chave com outro corpo é recusada
Limite de requisições em memória: com duas réplicas, o limite dobrava	Limite distribuído no Redis (janela fixa em Lua) e limitador global por IP	Teste de integração com duas instâncias da API compartilhando o contador
Toda leitura da vitrine ia ao banco	HybridCache: memória (5 s) + Redis (10 s) com invalidação por tag; a compra nunca lê do cache	Teste de integração: alteração do organizador aparece na vitrine; k6 na vitrine
Sem métricas nem rastreamento distribuído	OpenTelemetry: traces da requisição até o Worker (traceparent na outbox e no RabbitMQ), métricas de negócio, Grafana local e X-Ray/CloudWatch na AWS	Teste de integração verifica o traceparent gravado na outbox; Grafana no ambiente local
Trocar a chave JWT derrubava todas as sessões	Rotação com chaves anteriores aceitas; janela de 10 s para renovações simultâneas de várias abas	Testes de integração de rotação e de renovação simultânea
Tabelas crescendo sem limite	Limpeza periódica de outbox publicada, chaves de idempotência e sessões vencidas	Teste de integração com o relógio adiantado
Sem ambiente de produção	Terraform na AWS: ECS Fargate, RDS Multi-AZ, ElastiCache, Amazon MQ, WAF, KMS, Secrets Manager, alarmes e backups com cópia em outra região	terraform fmt, validate, tfint e checkov no CI (não aplicado numa conta real)
Sem plano para incidentes	Modelo de ameaças STRIDE, SLOs, quatro runbooks e plano de recuperação	Revisão; a restauração regional não foi exercitada
Interface sem testes ponta a ponta	Playwright (desktop e celular) no CI contra o ambiente completo	Três cenários: compra, sala de espera e rotas protegidas

Como funciona a fila virtual

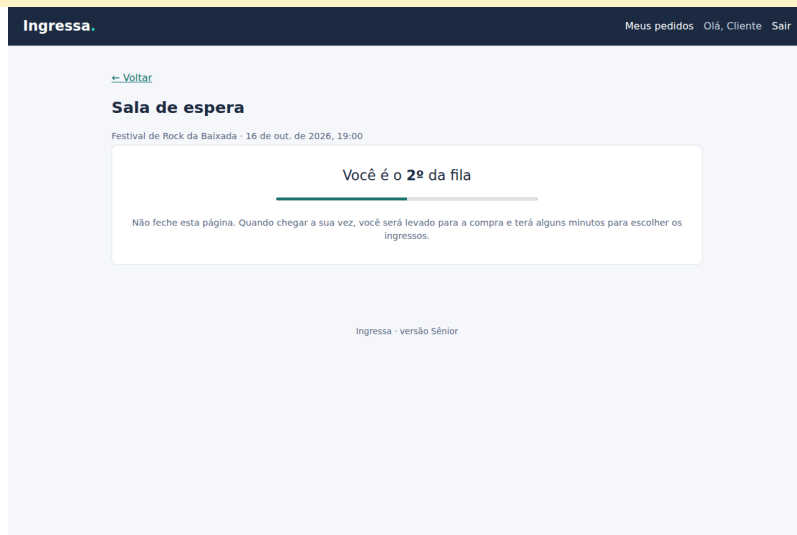
O problema não é só de desempenho, é de **justiça**: quem chegou primeiro deve comprar primeiro, e o banco só deve receber a quantidade de compradores que consegue atender.

- **Entrar:** o cliente é inserido num *sorted set* do evento com um número de sequência — a posição é a ordem de chegada. Consultar a posição não toca o banco.
- **Admitir:** a cada 2 segundos o Worker move da espera para os "ativos" apenas o número de pessoas que falta para chegar ao limite de compradores simultâneos, e emite um **passe** válido por 10 minutos.
- **Reservar:** a API consome o passe (uso único, ligado ao usuário) antes de reservar. Se a reserva falhar por regra de negócio, o passe é devolvido; se der certo, a vaga é liberada para o

próximo da fila.

- **Por que Redis:** cada uma dessas operações é um script Lua — o Redis executa de forma atômica, então duas réplicas da API ou do Worker nunca admitem a mesma pessoa duas vezes nem passam do limite.

Trade-off assumido: em eventos com fila, o Redis passa a ser dependência obrigatória da compra. Se ele cair, esses eventos param de vender. Foi uma escolha consciente: é melhor parar do que abrir a porta para todo mundo ao mesmo tempo. Nos demais eventos, o Redis fora do ar só desliga o cache e os limites (o sistema continua vendendo).



Sala de espera: posição na fila atualizada automaticamente, sem consultar o banco de dados.

Idempotência: o que acontece quando o cliente tenta de novo

Situação	Resposta da API
Primeira requisição	Executa normalmente
Mesma chave, mesmo conteúdo, já concluída	Devolve a resposta original com o cabeçalho Idempotent-Replayed
Mesma chave enquanto a primeira ainda executa	409 (em andamento)
Mesma chave com outro conteúdo	422 (chave reutilizada)
Requisição falhou	A chave é liberada para nova tentativa
Processo caiu no meio	Depois de 2 minutos a chave pode ser retomada, sem esperar a limpeza diária

Observabilidade e operação

- **Traces** ligando a requisição HTTP, as consultas SQL, o Redis, a mensagem no RabbitMQ e o processamento no Worker — o mesmo identificador volta no cabeçalho X-Trace-Id.
- **Métricas de negócio** (reservas, pagamentos, tamanho da fila, pessoas liberadas) ao lado das métricas técnicas.
- **SLOs** declarados e alarmes correspondentes: erros 5xx, latência p95, instâncias sem saúde, CPU e espaço do banco, memória do Redis e mensagens paradas na fila de falhas.
- **Runbooks** para restauração do banco (inclusive desastre regional), rotação da chave JWT, mensagens com falha e abertura de vendas com fila.
- **Painel do administrador** para listar e reprocessar mensagens que não conseguiram ser publicadas.

Resultados medidos

Teste de carga com k6, com **tudo rodando no mesmo notebook** (Intel i5-1135G7, 8 GB): banco, cache, fila, quatro instâncias da aplicação e o próprio gerador de carga.

Cenário	Resultado	Meta
Vitrine (leitura)	31.116 requisições em 1m45s (294,6 req/s), zero falhas , p95 de 66 ms	300 req/s, erros < 1%, p95 < 300 ms
Pico de vendas	300 compradores pela fila, 300 reservas, zero erros 5xx, nenhuma venda além da capacidade , nenhuma reserva sem passe	todos atendidos, sem 5xx, sem estouro de estoque
Latência sob pico	p95 de 2,0 s na fila e 4,3 s na reserva — acima da meta	< 200 ms e < 500 ms em produção

Por que a latência não bate a meta: 300 usuários virtuais, quatro instâncias, banco, cache e fila dividindo quatro núcleos com o gerador de carga. A *correção* se sustenta nesse cenário (zero erro, zero venda a mais); a *latência* só seria medível num ambiente onde o teste não disputa CPU com o sistema testado. As metas seguem valendo como SLO de produção.

O que o teste de carga encontrou

A primeira execução com 300 compradores devolveu **103 erros 500**. O log dos containers mostrou 53300: sorry, too many clients already: o PostgreSQL aceita 100 conexões e cada instância mantinha um pool que vai até 100 — com quatro instâncias, o pico pedia 400. A correção foi **orçar conexões** (teto por instância e limite maior no banco), e não aumentar o banco no escuro; o mesmo cálculo está documentado para a AWS. Depois disso, a mesma execução foi a zero erros. É o tipo de defeito que nenhum teste de unidade encontraria: só aparece com várias instâncias sob carga.

Produção na AWS (descrita em Terraform)

Internet → WAF → ALB (TLS 1.3) → ECS Fargate (API, Worker e interface, com autoscaling e rollback automático) → RDS PostgreSQL Multi-AZ, ElastiCache Redis e Amazon MQ, todos em sub-redes privadas e com TLS. Senhas geradas pelo Terraform e guardadas no Secrets Manager; migrations aplicadas por uma tarefa separada antes do deploy; backups diários com cópia para outra região.

Honestidade sobre o escopo: a infraestrutura foi escrita e verificada por ferramentas (fmt, validate, tfint, checkov), mas **não foi aplicada numa conta AWS real** — aplicar geraria custo. O que foi executado de verdade: todo o ambiente em Docker Compose com réplicas, os testes de integração contra PostgreSQL e Redis reais, os testes ponta a ponta e os testes de carga.

Decisão medida: continuar sem microsserviços

O plano previa extrair a fila virtual para um serviço próprio se a medição justificasse. Depois de implementá-la, a conclusão foi **não extrair**: o estado da fila vive no Redis e a admissão roda no Worker, que já escala separado da API — um serviço a mais só acrescentaria uma chamada de rede ao caminho da compra. A decisão, com os motivos, está registrada no ADR 0013.

7. Decisões arquiteturais

Cada decisão tem um registro próprio (ADR) no repositório. O resumo:

Decisão	Alternativas consideradas	Vantagens	Desvantagens / trade-off
Monólito simples na Júnior (ADR 0001)	Clean Architecture desde o início; microsserviços	Fácil de ler; um comando para rodar	Disciplina depende de convenção
SQLite na Júnior (ADR 0002)	PostgreSQL; LocalDB	Zero instalação	Esconde concorrência; não agrega decimal
JWT próprio (ADR 0003)	ASP.NET Identity; Auth0/Keycloak	Mecanismo visível e testado	Recursos extras precisam ser construídos
Camadas + Worker, sem microsserviços (ADR 0004)	Microsserviços; manter um projeto	Transação única; domínio testável	Mais projetos; banco compartilhado
UPDATE atômico para estoque (ADR 0005)	Lock pessimista; otimista no setor; lock no Redis	Correto sem travas longas	Não serve para assento numerado
Outbox + RabbitMQ (ADR 0006)	Processar na requisição; fila em memória; publicar após commit; outbox sem broker	Nenhuma mensagem perdida; filas de falha	Entrega "pelo menos uma vez"; mais um serviço
Refresh token em cookie (ADR 0007)	JWT longo no localStorage; sessão no servidor; BFF	Resistente a XSS; revogável	Exige mesma origem e lógica de renovação
Repositórios + consultas (ADR 0008)	DbContext na Application; repositório genérico	Application sem EF; leituras eficientes	Mais interfaces
Fila virtual no Redis com Lua (ADR 0009)	Fila no PostgreSQL; RabbitMQ; só aumentar recursos; serviço pronto (Cloudflare/Queue-it)	Posição em memória; operações atômicas entre réplicas; ordem justa	Redis vira dependência obrigatória nesses eventos
Idempotency-Key (ADR 0010)	Confiar nas regras de domínio; desabilitar o botão no front	Retentativa segura; padrão de mercado	Mais uma tabela; janela de falha entre commit e resposta
HybridCache memória + Redis (ADR 0011)	Sem cache; só memória; só Redis; CDN	Sem stampede; consistente entre réplicas	Vitrine pode atrasar até 5 s numa réplica
OpenTelemetry (ADR 0012)	Só logs; SDK de um fornecedor	Padrão aberto; destino por configuração	Custo de ingestão; exige ajuste de amostragem
Continuar sem microsserviços (ADR 0013)	Extrair fila virtual; extrair emissão de ingressos	Um modelo de dados, deploy simples; Worker já escala sozinho	Falha grave na API afeta tudo (mitigada por rollback automático)
ECS Fargate (ADR 0014)	EC2 com compose; EKS (Kubernetes); App Runner	Sem servidores nem plano de controle; integra com ALB, IAM e CloudWatch	Dependência da AWS; vCPU mais cara que EC2

8. Roadmap

Etapas	Entregas	Status
0. Análise	Avaliação das 27 ideias; escolha e ajuste do problema	Concluída
1. Júnior	API, autenticação, CRUD, vitrine, compra, testes, interface simples	Concluída
2. Pleno	Camadas, PostgreSQL, concorrência, reserva e pagamento, outbox, RabbitMQ, sessão segura, React, Docker, CI	Concluída (validação final no ambiente local)

Etapa	Entregas	Status
3. Sênior — medir	SLOs declarados e testes de carga com k6 (vitrine e pico de vendas)	Concluída
4. Sênior — escalar	Fila virtual no Redis, idempotência, limites distribuídos, cache híbrido, réplicas	Concluída
5. Sênior — operar	OpenTelemetry, Terraform/AWS, backups, runbooks, STRIDE, segurança no CI	Concluída (infraestrutura não aplicada numa conta real)
6. Próximos passos	Provedor de pagamento real, anti-cambista (CAPTCHA/limite por CPF), aplicar o Terraform em homologação, CDN	Fora do escopo das três versões

9. Aprendizados

Versão	Conhecimentos desenvolvidos
Júnior	Modelagem relacional com EF Core; API REST e códigos HTTP; validação; tratamento centralizado de erros; hash de senha; JWT; testes com banco em memória; TimeProvider; Git com commits semânticos; documentação
Pleno	Arquitetura em camadas e inversão de dependência; domínio rico; transações e níveis de consistência; condição de corrida e concorrência otimista; PostgreSQL (xmin, índices parciais, SKIP LOCKED, CHECK); mensageria (exchanges, filas quorum, dead letter, confirmações, idempotência); padrão Outbox; sessão segura (cookies, CSRF, XSS, rotação); limite de requisições; Docker multi-stage; CI; Testcontainers; React e TypeScript
Sênior	Redis além de cache (sorted sets e scripts Lua atômicos); controle de admissão e filas virtuais; idempotência de requisições; limites distribuídos; cache híbrido e invalidação; OpenTelemetry (traces, métricas e propagação de contexto por mensageria); SLOs e alarmes; testes de carga com k6; Playwright; Terraform e ECS Fargate; modelagem de ameaças STRIDE; runbooks e plano de recuperação; e a disciplina de descartar uma ideia (microserviços) com justificativa registrada

10. Valor para o portfólio

O que um recrutador técnico consegue verificar no repositório:

Competência	Evidência
Constrói uma API completa	Versão Júnior rodando com um comando; 16 endpoints documentados
Escreve código testável	Mais de 290 testes automatizados em seis níveis (domínio, casos de uso, integração com PostgreSQL e Redis reais, scripts Lua, interface e ponta a ponta)
Mede em vez de supor	k6 com metas que reprovam a execução; a medição encontrou um limite de conexões do banco que virou correção documentada
Entende concorrência	Experimento reproduzível (200 × 10) e teste com 40 clientes HTTP
Pensa em falhas	Outbox, idempotência, estorno automático, dead letter, expiração
Pensa em segurança	Sessão resistente a XSS/CSRF, detecção de token roubado, bloqueio, limites, CSP
Aguenta pico de acesso	Fila virtual com controle de admissão, limites distribuídos, cache e testes de carga com metas que reprovam a execução
Opera o que constrói	OpenTelemetry ponta a ponta, SLOs, alarmes, runbooks, limpeza de dados e painel para reprocessar falhas
Sabe levar para a nuvem	Ambiente de produção descrito em Terraform (ECS Fargate, RDS, ElastiCache, Amazon MQ, WAF, backups), verificado no CI
Toma decisões e explica	14 ADRs com alternativas e trade-offs, incluindo uma ideia planejada e depois descartada por medição
Trabalha como em uma empresa	Conventional Commits, CI, Docker, README por versão, migrations verificadas
Consegue evoluir um sistema	Três versões no mesmo repositório, cada uma motivada pelas limitações da anterior

Pergunta de entrevista que este projeto responde bem: "Como você evitaria vender o mesmo ingresso duas vezes?" — com a resposta errada (Júnior), a prova de que ela é errada (experimento) e a resposta certa (Pleno), cada uma com código e teste.

Apêndice — como rodar e validar

Versão Júnior

```
cd ingressa/junior
dotnet test
dotnet run --project src/Ingressa.Api --launch-profile http
# http://localhost:5080 · cliente@ingressa.dev / Senha@123
```

Versão Pleno

```
cd ingressa/pleno
cp .env.example .env # troque as senhas
docker compose up --build
# interface: http://localhost:8080 API: http://localhost:5180/scalar
# RabbitMQ: http://localhost:15672 e-mails: http://localhost:8025

dotnet test # Docker aberto (testes de integração)
cd web && npm test
```

Versão Sênior

```
cd ingressa/senior
cp .env.example .env # troque as senhas
docker compose up --build
# interface: http://localhost:8080 Grafana: http://localhost:3000

dotnet test # PostgreSQL e Redis reais
cd web && npm test && npm run e2e # interface e ponta a ponta
docker compose --profile carga run --rm k6 run /scripts/pico-de-vendas.js
```

Nota sobre o processo de validação

O código foi escrito e revisado em um ambiente sem acesso ao repositório de pacotes .NET (NuGet). Para não depender apenas de revisão, as regras de domínio e os casos de uso foram compilados e executados ali com implementações mínimas das bibliotecas, o experimento de concorrência foi executado em um PostgreSQL 16 real, e a interface foi testada em um navegador automatizado (Playwright). A compilação com os pacotes oficiais, a geração das migrations e os testes de integração com Testcontainers são executados no computador de desenvolvimento e no CI do GitHub. Na versão Sênior, os scripts Lua da fila foram executados contra um Redis real, a interface foi verificada com Playwright e os testes de carga com k6 rodaram contra o ambiente completo em Docker Compose; os resultados estão em `senior/docs/desempenho.md`.